EXPERIMENT 9: IMPLEMENTATION OF BIT STUFFING MECHANISM USING C

PROGRAM:

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char data[200], stuffed[420];
    int i, j = 0, count = 0;
    printf("Enter data bits (0s and 1s only): ");
    scanf("%199s", data);
    strcpy(stuffed, "01111110");
    j = strlen(stuffed);

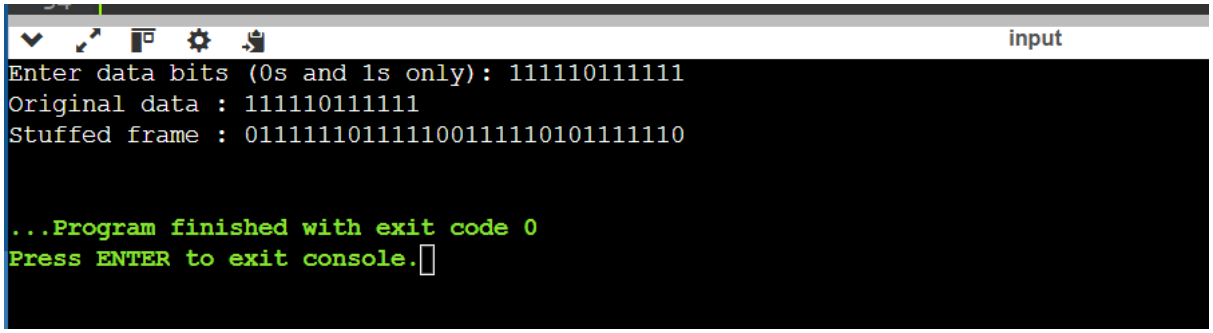
    for (i = 0; data[i] != '\0'; i++) {
        if (data[i] != '0' && data[i] != '1') {
            printf("Invalid input: enter only 0 and 1.\n");
            return 1;
        }

        stuffed[j++] = data[i];

        if (data[i] == '1') {
            count++;
            if (count == 5) {
                stuffed[j++] = '0';
                count = 0;
            }
        } else {
            count = 0;
```

```
        }
    }
    strcpy(stuffed + j, "01111110");
    printf("Original data : %s\n", data);
    printf("Stuffed frame : %s\n", stuffed);
    return 0;
}
```

OUTPUT :



EXPERIMENT 10: SIMULATION OF CRC (CYCLIC REDUNDANCY CHECK) USING C

PROGRAM:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char data[100], divisor[30], temp[140], codeword[140];
    int n, m, i, j, error = 0;

    printf("Enter data bits: ");
    scanf("%99s", data);
    printf("Enter generator bits (e.g. 1011): ");
    scanf("%29s", divisor);

    n = (int)strlen(data);
    m = (int)strlen(divisor);

    if (m < 2 || m > n + 1 || divisor[0] != '1') {
        printf("Invalid generator or generator length.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        if (data[i] != '0' && data[i] != '1') {
            printf("Data must contain only 0 and 1.\n");
            return 1;
        }
    }
    for (i = 0; i < m; i++) {
```

```
    if (divisor[i] != '0' && divisor[i] != '1') {
        printf("Generator must contain only 0 and 1.\n");
        return 1;
    }
}

strcpy(temp, data);
for (i = 0; i < m - 1; i++)
    temp[n + i] = '0';
temp[n + m - 1] = '\0';

for (i = 0; i < n; i++) {
    if (temp[i] == '1') {
        for (j = 0; j < m; j++)
            temp[i + j] =
                (temp[i + j] == divisor[j]) ? '0' : '1';
    }
}

strcpy(codeword, data);
strncat(codeword, temp + n, m - 1);

printf("\nOriginal data : %s\n", data);
printf("Generator     : %s\n", divisor);
printf("CRC remainder : %.*s\n", m - 1, temp + n);
printf("Codeword      : %s\n", codeword);

/* Receiver verification */
strcpy(temp, codeword);
```

```
    for (i = 0; i < n; i++) {
        if (temp[i] == '1') {
            for (j = 0; j < m; j++)
                temp[i + j] =
                    (temp[i + j] == divisor[j]) ? '0' : '1';
        }
    }

    for (i = n; i < n + m - 1; i++)
        if (temp[i] != '0')
            error = 1;

    if (error)
        printf("Receiver: Error detected.\n");
    else
        printf("Receiver: No error detected by CRC.\n");

    return 0;
}
```

OUTPUT:



EXPERIMENT 11: IMPLEMENTATION OF SLIDING WINDOW PROTOCOL USING C

PROGRAM:

```
#include <stdio.h>
```

```
int main(void) {
    int total, window, base = 0, i, end;

    printf("Enter total number of frames: ");
    if (scanf("%d", &total) != 1) return 1;

    printf("Enter window size: ");
    if (scanf("%d", &window) != 1) return 1;

    if (total <= 0 || window <= 0) {
        printf("Enter positive values only.\n");
        return 1;
    }

    printf("\nSliding Window Protocol Simulation\n");

    while (base < total) {
        end = base + window;
        if (end > total)
            end = total;

        printf("\nSender window: frames %d to %d\n",
               base, end - 1);

        for (i = base; i < end; i++)
            printf("Sender: Sending frame %d\n", i);

        for (i = base; i < end; i++) {
```

```
            printf("Receiver: Frame %d received\n", i);
            printf("Receiver: ACK %d sent\n", i);
        }

        base = end;
        printf("Sender: Window moved forward\n");
    }

    printf("\nAll frames sent and acknowledged.\n");
    return 0;
}
```

OUTPUT:

